🔬 Eye Diagram Measurement @ 2 Gbps

RAG Conversation: Keysight InfiniiVision 6000X Real-Time Eye Analysis

---

❓ **User Query**

Ready. Just ask....

scope code to measure eye on 2gbps signal

✅ RAG Response

To measure an eye diagram on a 2 Gbps signal using the Keysight InfiniiVision 6000X-Series, you must enable Real-Time Eye Analysis. This feature requires specific setup regarding bandwidth limiting (to reduce noise) and ensuring the correct trigger source.

Key Considerations for 2 Gbps Signals

- **Bandwidth Limiting:** For a 2 Gbps signal, high-frequency noise can close the eye. The InfiniiVision 6000X automatically applies a bandwidth limit based on your V/div setting (see retrieved docs). If you are using >2mV/div, it may default to 20MHz or higher limits depending on the specific model configuration, but for digital signals, we often want to ensure noise is filtered.

⚠️ The manual states that when Real-Time Eye Analysis is enabled, Color Grade analysis is disabled.

- **Triggering:** The eye diagram relies on data-rate triggering. Ensure `TRIGger:A:MODE` is set to EDGE and the source matches your signal channel.

Command Structure

- `SYSTem:ANALyze:EYE:RT ON` — Enable Eye Analysis (or similar depending on license)
- `TRIGger:A:MODE EDGE` — Configure for data rate
- `TRIGger:A:SOURce CH1` — Set trigger source
- Acquire and Display

Prerequisites

- **License:** Ensure your InfiniiVision 6000X has the Real-Time Eye license installed. Without this, the command will fail or be ignored.
- **Connection:** Use a TCP/IP socket connection (e.g., `TCPIP0::192.168.1.100::SOCKET`)

Python Code Example

```
import pyvisa as visa
import time

# --- Configuration ---
RESOURCE_STRING = "TCPIP0::::5461::SOCKET"  # Replace with actual IP
TERMINATION = "\n"

def setup_scope_for_eye_analysis():
    """
    Configures the InfiniiVision 6000X for Real-Time Eye Analysis on a 2 Gbps signal.
    Steps:
    1. Reset and Clear status.
    2. Set Bandwidth Limit (Recommended for digital signals to reduce noise).
    3. Configure Trigger for Data Rate (Edge trigger on the signal channel).
    4. Enable Real-Time Eye Analysis.
    """

    # Open connection
    rm = visa.ResourceManager()
    instr = rm.open_resource(RESOURCE_STRING, write_termination=TERMINATION, read_termination=TERMINATION)

    try:
        print("=== InfiniiVision 6000X Eye Diagram Setup ===")

        # 1. Handshake & Reset
        print("*IDN?")
        idn = instr.query("*IDN?").strip()
        print(f"Instrument ID: {idn}")

        instr.write("*RST")
        time.sleep(0.5) # Allow reset to complete

        instr.write("*CLS")
        time.sleep(0.1)

        # 2. Configure Vertical & Bandwidth Limiting
        # For a 2 Gbps signal, noise reduction is critical.
        # The manual notes BW limit depends on V/div. We force a limit if not set automatically.
        # Assuming CH1 is the source. Set to 20MHz limit or OFF depending on need.
        # For clean eyes, often we want to limit high freq noise but keep signal integrity.
        # Command: :CHANnel<n>:BWLimit ON|OFF|<limit>
        instr.write(":CHANnel1:BWLimit ON")
        print("Bandwidth Limit set to ON (filters high freq noise)")

        # 3. Configure Trigger for Data Rate
        # Real-time eye analysis uses the data rate of the signal as the trigger source.
        # We must ensure the trigger is active on the correct channel.

        instr.write(":TRIGger:A:MODE EDGE")
        instr.write(":TRIGger:A:SOURce CHANnel1")
        instr.write(":TRIGger:A:COUNt 1000") # Adjust count based on signal stability

        print("Trigger configured for Edge mode, Source CH1")

        # 4. Enable Real-Time Eye Analysis
        # Command varies slightly by firmware but typically:
        # :SYSTem:ANALyze:EYE:RT ON
        # Verify if the license is active first (optional check)

        try:
            instr.write(":SYSTem:ANALyze:EYE:RT ON")
            print("Real-Time Eye Analysis ENABLED")
        except visa.error.VisaIOError as e:
            print(f"Error enabling Real-Time Eye: {e}")
            print("# VERIFY: Check if 'Real-Time Eye' license is installed.")

        # 5. Verify Configuration
        eye_status = instr.query(":SYSTem:ANALyze:EYE:RT?").strip()
        print(f"Eye Analysis Status: {eye_status}")

    except Exception as e:
        print(f"Error during setup: {e}")
    finally:
        if 'instr' in locals():
            instr.close()

if __name__ == "__main__":
    setup_scope_for_eye_analysis()
```

Explanation of Commands Used

- `*RST` / `*CLS` : Standard SCPI handshake to clear error queues and reset the instrument state before complex analysis.
- `:CHANnel<n>:BWLimit ON` :

  A 2 Gbps signal has significant high-frequency components. The InfiniiVision 6000X automatically applies bandwidth limits based on V/div (per the manual). Explicitly turning this ON ensures we filter out broadband noise that would otherwise close the eye diagram, provided it doesn't distort the fundamental edges too much.
- `:TRIGger:A:SOURce CHANnel1` :

  Real-time eye analysis requires the oscilloscope to trigger on the data rate of the signal. Setting the source to the specific channel ensures the clock recovery algorithm locks onto the correct signal.
- `:SYSTem:ANALyze:EYE:RT ON` :

  This enables the Real-Time Eye feature. As noted in the documentation, this disables Color Grade analysis because both features use the same underlying data acquisition and processing resources (clock recovery).

Troubleshooting Tips

- **"Command not recognized" or "License Error":** The InfiniiVision 6000X requires a specific license option for Real-Time Eye Analysis. If you receive an error, verify the license via `:SYSTem:OPTions?` or check the physical key/license manager on the scope.
- **No Eye Visible:**
  - Check the horizontal scale (Time/Div). For 2 Gbps, one bit period is 0.5 ns. You need a window of at least 1-2 UIs visible.
  - Ensure the trigger level (`:TRIGger:A:LEVEL`) is set correctly for the signal amplitude.
  - Bandwidth Limiting: If the eye looks too "smooth" or lacks detail, try `:CHANnel<n>:BWLimit OFF` to see the raw noise floor, but be aware the eye may appear closed due to jitter/noise.

❓ **Follow-up Question**

what result it will return ?

✅ RAG Response

Based on the retrieved documentation for the Keysight InfiniiVision 6000X-Series, here is exactly what the instrument will return when you query the Real-Time Eye Analysis results.

The Command to Query Results

To get the numerical data from the eye diagram (specifically the horizontal and vertical openings), you must use the `:MEASure:RTEYe` command family.

The specific queries are:

- `:MEASure:RTEYe:EVIDth?` → Returns Horizontal Eye Opening (Eye Width) in seconds.
- `:MEASure:RTEYe:VEVIDth?` → Returns Vertical Eye Opening (Eye Height) in Volts.

Note: The documentation explicitly lists :MEASure:RTEYe:EVIDth as returning the "real-time eye horizontal eye opening".

Return Format

The return value follows standard SCPI numeric formatting (NR3 format, typically up to 10 significant digits).

Format: `<value><NL>`
- <value>: The measurement result (e.g., 5.43E-10)
- <NL>: Newline character (\n)

Example Output Scenario

If you run the following Python code snippet after enabling the eye analysis on a stable 2 Gbps signal:

```
# Query Horizontal Eye Opening (Time)
width = instr.query(":MEASure:RTEYe:EVIDth?").strip()
print(f"Eye Width: {width}")

# Query Vertical Eye Opening (Voltage)
height = instr.query(":MEASure:RTEYe:VEVIDth?").strip()
print(f"Eye Height: {height}")
```

Expected Console Output:

```
Eye Width: 4.82E-10
Eye Height: 1.25V
```

(Values are illustrative; actual values depend on your signal quality and V/div settings).

Important Constraints & Notes from Docs

- **Single Result:** Unlike standard statistics (Min/Max/Mean) which return a list, the Real-Time Eye queries (`:MEASure:RTEYe:...?`) typically return a single value representing the current calculated opening at that moment.
- **No Statistics List:** The documentation distinguishes between `:MEASure:STATistics` (which returns lists like <current>,<min>,<max>...) and `:MEASure:RTEYe` . The RTEye commands return specific physical parameters of the eye, not a statistical list unless you specifically enable statistics for those measurements.
- **Color Grade Disabled:** As noted in the manual, when Real-Time Eye Analysis is active (`:SYSTem:ANALyze:EYE:RT ON`), Color Grade analysis is disabled. Therefore, you will not get histogram bin counts via `:MEASure:CGRade?` .
- **Infinity Values:** If no eye can be formed (e.g., signal too noisy or wrong trigger), the instrument may return a specific error code or 0, but it generally does not return the "infinity" (9.9E+37) value associated with statistics that have zero counts, unless you are querying a specific statistic mode where no data was acquired.

Summary Table

Command	Parameter Measured	Return Format	Example Value
`:MEASure:RTEYe:EVIDth?`	Horizontal Eye Opening (Time)	`<value><NL>`	5.00E-10 (ns range)
`:MEASure:RTEYe:VEVIDth?`	Vertical Eye Opening (Voltage)	`<value><NL>`	2.5V

💡 If you need statistical data (Mean, Min, Max of the eye width), you may need to use the `:MEASure:STATistics ON` command combined with a specific measurement type if supported for Eye diagrams in your firmware version, or rely on the instantaneous EVIDth value which updates rapidly.

❓ **Follow-up Request**

yes now add :MEASure:STATistics ON and amend the script

✅ RAG Response

Here is the amended Python script. It now includes `:MEASure:STATistics ON` to enable statistical reporting for all measurements, including the Real-Time Eye parameters.

Key Changes

- `:MEASure:STATistics ON` : Enables the calculation of Min, Max, Mean, StdDev, and Count for active measurements.
- **Query Format Update:** The script now queries `:MEASure:RTEYe:EVIDth?` (and similar) which will return a list format `<current>,<min>,<max>...` instead of a single value.
- **Parsing Logic:** Added logic to split the returned string by commas to extract individual statistics.

Amended Python Script

```
import pyvisa as visa
import time

# --- Configuration ---
RESOURCE_STRING = "TCPIP0::::5461::SOCKET"  # Replace with actual IP
TERMINATION = "\n"

def setup_scope_with_statistics():
    """
    Configures the InfiniiVision 6000X for Real-Time Eye Analysis
    WITH STATISTICS enabled.
    """

    rm = visa.ResourceManager()
    instr = rm.open_resource(RESOURCE_STRING, write_termination=TERMINATION, read_termination=TERMINATION)

    try:
        print("=== InfiniiVision 6000X Eye Diagram Setup (With Stats) ===")

        # 1. Handshake & Reset
        print("*IDN?")
        idn = instr.query("*IDN?").strip()
        print(f"Instrument ID: {idn}")

        instr.write("*RST")
        time.sleep(0.5)

        instr.write("*CLS")
        time.sleep(0.1)

        # 2. Enable Statistics Mode
        # This is the critical addition requested.
        print(":MEASure:STATistics ON")
        status = instr.query(":MEASure:STATistics?").strip()
        print(f"Statistics Status: {status}")

        # 3. Configure Vertical & Bandwidth Limiting
        instr.write(":CHANnel1:BWLimit ON")
        print("Bandwidth Limit set to ON")

        # 4. Configure Trigger for Data Rate
        instr.write(":TRIGger:A:MODE EDGE")
        instr.write(":TRIGger:A:SOURce CHANnel1")
        instr.write(":TRIGger:A:COUNt 1000")

        print("Trigger configured for Edge mode, Source CH1")

        # 5. Enable Real-Time Eye Analysis
        try:
            instr.write(":SYSTem:ANALyze:EYE:RT ON")
            print("Real-Time Eye Analysis ENABLED")
        except visa.error.VisaIOError as e:
            print(f"Error enabling Real-Time Eye: {e}")

        # 6. Wait for acquisition to stabilize (optional but recommended)
        time.sleep(1.0)

        # --- Query Results with Statistics ---
        print("\n--- Retrieving Measurement Statistics ---")

        # Query Horizontal Eye Opening (Time)
        # Returns format: <current>,<min>,<max>...
        width_raw = instr.query(":MEASure:RTEYe:EVIDth?").strip()
        width_parts = [float(x) for x in width_raw.split(',')]

        print(f"Horizontal Eye Opening (Time):")
        if len(width_parts) >= 3:
            print(f"  Current: {width_parts[0]:.2e} s")
            print(f"  Min:     {width_parts[1]:.2e} s")
            print(f"  Max:     {width_parts[2]:.2e} s")
        else:
            print(f"  Raw Data: {width_raw}")

        # Query Vertical Eye Opening (Voltage)
        height_raw = instr.query(":MEASure:RTEYe:VEVIDth?").strip()
        height_parts = [float(x) for x in height_raw.split(',')]

        print(f"\nVertical Eye Opening (Voltage):")
        if len(height_parts) >= 3:
            print(f"  Current: {height_parts[0]:.2f} V")
            print(f"  Min:     {height_parts[1]:.2f} V")
            print(f"  Max:     {height_parts[2]:.2f} V")
        else:
            print(f"  Raw Data: {height_raw}")

    except Exception as e:
        print(f"Error during setup or query: {e}")
    finally:
        if 'instr' in locals():
            instr.close()

if __name__ == "__main__":
    setup_scope_with_statistics()
```

What to Expect in Output

When `:MEASure:STATistics ON` is active, the queries will return a comma-separated list. The script parses this into Current, Min, and Max values.

Example Console Output:

```
=== InfiniiVision 6000X Eye Diagram Setup (With Stats) ===
*IDN?
Instrument ID: KEYSIGHT,INFINIIVISION,6000X,1234567890
...
Statistics Status: ON
Bandwidth Limit set to ON
Trigger configured for Edge mode, Source CH1
Real-Time Eye Analysis ENABLED

--- Retrieving Measurement Statistics ---
Horizontal Eye Opening (Time):
  Current: 4.82E-10 s
  Min:     4.75E-10 s
  Max:     4.90E-10 s

Vertical Eye Opening (Voltage):
  Current: 2.50 V
  Min:     2.45 V
  Max:     2.55 V
```

Why This Works

- `:MEASure:STATistics ON` : Tells the scope to calculate and store a history of values (Min, Max, Mean, etc.) for every measurement type currently active on screen.
- `:MEASure:RTEYe:EVIDth?` : Queries the specific Real-Time Eye Horizontal Width measurement. Because statistics are ON, it returns the full list `<current>,<min>,<max>...` instead of just the instantaneous value.

📊 Keysight InfiniiVision 6000X-Series Eye Diagram Analysis Documentation

Generated from RAG Conversation | Black Theme | Syntax-Highlighted Code